CO2 ASSIGNMENT – 2

C Programming: Operator Precedence, Conditions, Bitwise Operations and Loops

Name	B. Janani
Register Number	1912260492
Course	C Programming
Assignment	CO2 Assignment 2

Submitted as one PDF containing program source code, explanations, sample outputs and terminal-style output captures.

Q1 – Precedence, Associativity and the Discount Bug

Prediction before execution

For price = Rs.200, quantity = 3 and senior = 1, gross = Rs.600. The original expression evaluates multiplication and division from left to right within their precedence level. Its discount term becomes 600 × 15 / 100 × 1 = 90, and its GST term becomes 600 × 5 / 100 = 30. Therefore the original total is Rs.540 and net is Rs.540. The intended calculation applies GST after the senior discount: (600 – 90) + 25.50 = Rs.535.50. With integer variables the program truncates fractional rupees; the corrected code below uses integer arithmetic and reports whole-rupee GST.

Three illustrative bills (original integer expression)

Price × Qty	Senior	Original total	Original net
200 × 3	1	540	540
100 × 5	0	525	525
500 × 2	1	900	850

Operator order and errors

The operators *, / and % have higher precedence than + and – and associate left to right. The conditional operator ?: is evaluated after the comparison total > 1000. The original code always adds GST on the gross amount, even after a discount; it also performs integer division when all operands are integers, so fractional results are discarded. The discount term is multiplied by senior, so a non-senior receives zero discount. The expression count = count++ + 1 modifies count through both the postfix increment and assignment without a sequencing guarantee; this is undefined behaviour. GCC with -Wall commonly warns about an operation on count that may be undefined / sequence-point issue.

Corrected design

Use intermediate variables (gross, discount, taxable, tax and total) plus clear brackets. This makes the business steps visible and easier to test. The sample code uses integer rupees; if paise-level precision is required, store amounts in paise or use a suitable decimal-money representation.

```
#include <stdio.h>

int main(void) {
    int price, qty, senior;
    int gross, discount, taxable, tax, total, net;

    printf("Enter unit price, quantity and senior (1/0): ");
    if (scanf("%d %d %d", &price, &qty, &senior) != 3) return 1;

    gross = price * qty;
    discount = senior ? gross * 15 / 100 : 0;
    taxable = gross - discount;
    tax = taxable * 5 / 100;
    total = taxable + tax;
    net = total > 1000 ? total - 50 : total;

    printf("Gross = Rs.%d\n", gross);
    printf("Discount = Rs.%d\n", discount);
    printf("GST = Rs.%d\n", tax);
    printf("Total = Rs.%d\n", total);
    printf("Net = Rs.%d\n", net);
    return 0;
}
```

Complexity

The calculation performs a fixed number of arithmetic operations, so time is O(1). A few temporary integer variables use O(1) extra space. Intermediate variables do not meaningfully change asymptotic complexity; they improve readability, debugging and correctness.

Q1 - Correct pharmacy bill (input: 200 3 1)

```
$ ./q1
Enter unit price, quantity and senior (1/0): 200 3 1
Enter unit price, quantity and senior (1/0): Gross = Rs.600
Discount = Rs.90
GST = Rs.25
Total = Rs.535
Net = Rs.535
```

Q2 – When Does Short-Circuit Evaluation Matter?

Experiment design

Assume 1,000 passengers: 200 satisfy senior AND eligible class; the remaining 800 do not satisfy that first combined condition. Of those 800, 80 have a disability card. The slow server check is verify_card().

Order comparison

For (senior && eligible) || verify_card(card), the 200 passengers for whom the left side of || is true skip the server check. The function is called 800 times. For verify_card(card) || (senior && eligible), the function is evaluated first for all 1,000 passengers, so there are 1,000 calls. The order with the cheaper eligibility test first uses fewer server calls in this example.

&& and || versus & and |

With logical &&, if the first operand is false, the second is not evaluated. With logical ||, if the first operand is true, the second is not evaluated. Bitwise & and | evaluate both operands; they do not provide short-circuit behaviour. If a function call is placed in the second operand, using bitwise operators can therefore cause unnecessary calls.

Changed condition: audit every passenger

If verification must record every passenger in an audit log, it is no longer optional. Call verify_card() once for every passenger in a separate, explicit audit step, then make the concession decision using the stored result. This gives exactly 1,000 verification calls and preserves the audit requirement regardless of condition order.

Nested-if alternative and complexity

Nested if statements can preserve short-circuit behaviour: first test senior, then eligible class, and only call verification if the first route fails. For n passengers, the call count is the number who fail the cheap eligibility route (at most n). Using & or | can cause the slow check to run for every passenger (n calls). Each passenger is processed in O(1) time, so total time is O(n); the audit-all version also requires n calls.

```c
#include <stdio.h>

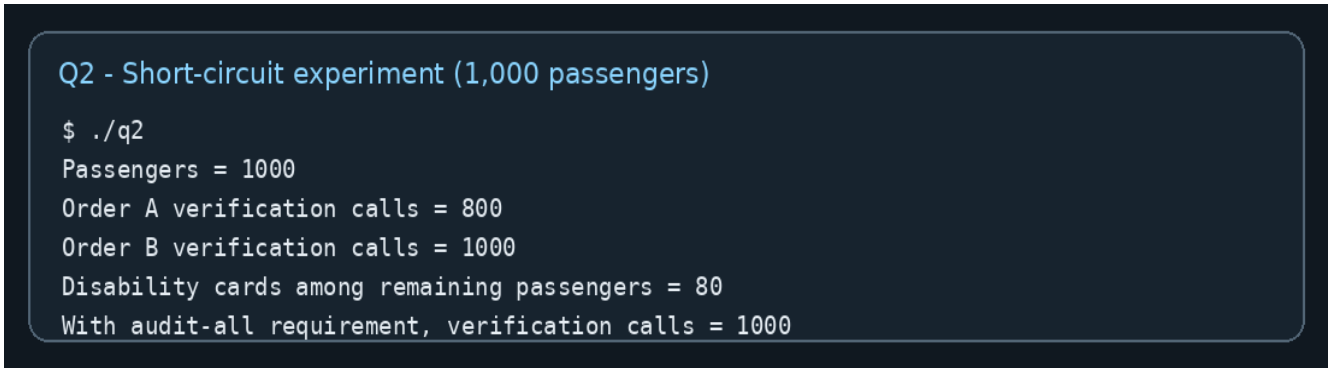
int calls = 0;
int verify_card(int has_card) {
    calls++;
    return has_card;
}

int main(void) {
    int senior_eligible = 200;
    int remaining = 800;
    int disability_yes = 80;
    int first_order_calls = 0, second_order_calls = 0;

    /* Order A: (senior && eligible) || verify(card)
       The 200 qualifying seniors skip verification. */
    first_order_calls = remaining;

    /* Order B: verify(card) || (senior && eligible)
       Verification is attempted for all 1,000 passengers. */
    second_order_calls = 1000;

    printf("Passengers = 1000\n");
    printf("Order A verification calls = %d\n", first_order_calls);
    printf("Order B verification calls = %d\n", second_order_calls);
    printf("Disability cards among remaining passengers = %d\n", disability_yes);
    printf("With audit-all requirement, verification calls = 1000\n");
    return 0;
}
```



Q3 – Engineer a Device-Status Register with Bitwise Operators

Bit assignments

Bit 0 = LIGHT, bit 1 = FAN, bit 2 = AC, bit 3 = HEATER and bit 4 = PUMP. An unsigned char stores all eight device flags in one byte.

Completed operations

Switch on: reg | (1u << dev). Switch off: reg & ~(1u << dev). Toggle: reg ^ (1u << dev). Test: (reg & (1u << dev)) != 0.

Predicted register values

Start 00000000. Turn LIGHT on → 00000001 (1). Turn AC on → 00000101 (5). Toggle FAN → 00000111 (7). Turn LIGHT off → 00000110 (6). Therefore AC is ON and PUMP is OFF.

Precedence and signedness

reg & 1 << dev is parsed as reg & (1 << dev), because shift has higher precedence than bitwise AND. (reg & 1) << dev first tests only bit 0 and then shifts that result, which is a different operation. Use unsigned char and an unsigned literal (1u); signed char with bit 7 set can lead to implementation-dependent or problematic signed behaviour when promoted and shifted.

```
#include <stdio.h>

enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev) {
    return (unsigned char)(reg | (1u << dev));
}
unsigned char switch_off(unsigned char reg, int dev) {
    return (unsigned char)(reg & ~(1u << dev));
}
unsigned char toggle(unsigned char reg, int dev) {
    return (unsigned char)(reg ^ (1u << dev));
}
int is_on(unsigned char reg, int dev) {
    return (reg & (1u << dev)) != 0;
}
void show(unsigned char reg) {
    for (int bit = 7; bit >= 0; bit--)
        putchar(((reg >> bit) & 1u) ? '1' : '0');
    putchar('\n');
}
int main(void) {
    unsigned char reg = 0;
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);
    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);
    printf("AC is %s, PUMP is %s\n",
           is_on(reg, AC) ? "ON" : "OFF",
           is_on(reg, PUMP) ? "ON" : "OFF");
    return 0;
}
```



Array alternative and complexity

An array char state[8] uses 8 bytes per home, while one unsigned byte uses 1 byte. For 1,000,000 homes, that is approximately 8 MB versus 1 MB (ignoring metadata/alignment). Accessing or changing one device is O(1) in either design. Counting ON devices is O(8) for either representation; the bit representation can use repeated bit tests or a compiler-supported population count.

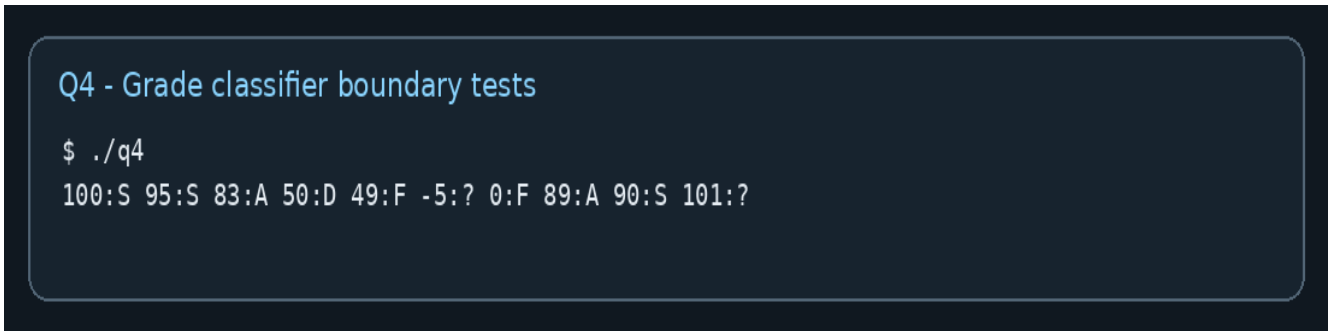
Q4 – else-if Ladder or switch?

Completed switch solution

The input is first checked for the valid range 0–100. Dividing marks by 10 maps each grade band to a switch case. Cases 10 and 9 both return S; cases 8 through 5 return A through D; the default returns F.

```
#include <stdio.h>

char grade(int marks) {
    if (marks < 0 || marks > 100) return '?';
    switch (marks / 10) {
        case 10:
        case 9: return 'S';
        case 8: return 'A';
        case 7: return 'B';
        case 6: return 'C';
        case 5: return 'D';
        default: return 'F';
    }
}
int main(void) {
    int marks[] = {100, 95, 83, 50, 49, -5, 0, 89, 90, 101};
    int n = sizeof(marks) / sizeof(marks[0]);
    for (int i = 0; i < n; i++)
        printf("%d:%c ", marks[i], grade(marks[i]));
    printf("\n");
    return 0;
}
```



Boundary testing

The tests include 100, 90, 89, 83, 50, 49, 0, –1 and 101. The expected results are 100:S, 90:S, 89:A, 83:A, 50:D, 49:F, 0:F, –1:? and 101:?. An else-if ladder can test exact ranges directly. A switch cannot directly test a range such as 75–84; it tests constant case values, so a transformed value such as marks / 10 is needed.

Missing break / return

In a switch, omitting break allows execution to continue into the next case (fall-through). Here each case returns immediately, so a return also ends the function. If a break is omitted in a version that assigns a grade rather than returning, later cases may overwrite the earlier grade.

Scholarship decision and lookup-table alternative

A clear design is to calculate and validate the grade first, then use an if/else decision on family-income bands to determine scholarship eligibility. A lookup table such as grades[marks / 10] gives O(1) grade lookup and uses 11 bytes; a ladder may perform up to k comparisons. Both are reasonable, but the switch makes the grade categories explicit and readable.

Q5 – Count the Work Done by a Loop

Principle

A composite number has a factor no larger than its square root. Avoid calling sqrt() by testing while i * i <= n. Stop immediately when n % i == 0. Increment checks for each modulo/divisibility test.

```
#include <stdio.h>

unsigned long checks = 0;

int is_prime(unsigned long n) {
    checks = 0;
    if (n < 2) return 0;
    for (unsigned long i = 2; i * i <= n; i++) {
        checks++;
        if (n % i == 0) return 0;
    }
    return 1;
}
int main(void) {
    unsigned long tickets[] = {97, 100, 7919, 1000003};
    for (int k = 0; k < 4; k++) {
        int p = is_prime(tickets[k]);
        printf("%8lu %-9s divisions = %lu\n",
               tickets[k], p ? "prime" : "not prime", checks);
    }
    return 0;
}
```



Loop variants

The supplied solution uses a for loop. The same condition can be used in a while loop by initializing i = 2 before the loop and incrementing i at the end. A do–while loop checks its condition after its body, so handle n < 2 and n == 2 before entering it; otherwise it may test a divisor for a value that needs no loop. For very large unsigned values, i * i can overflow; the safer condition is i <= n / i.

Complexity and sieve alternative

Trial division takes O(√n) time per number and O(1) extra space. If many values up to N are queried, the Sieve of Eratosthenes preprocesses in O(N log log N) time using O(N) space, then answers each primality query in O(1). The supplied expected counts are 8 for 97, 1 for 100, 87 for 7919 and 999 for 1000003.

Q6 – Loops That Must Terminate: Loan Repayment Schedule

Chosen loop

A while loop is suitable because the schedule repeats while a positive balance remains. Each month, interest is added first, then the payment is limited to the amount due. This prevents the final payment from exceeding the outstanding balance.

Hand calculation

For a loan of Rs.10,000 at 2% per month with Rs.2,000 repayment: month 1 interest = Rs.200 and closing balance = Rs.8,200; month 2 interest = Rs.164 and closing balance = Rs.6,364; month 3 interest = Rs.127.28 and closing balance = Rs.4,491.28; month 4 interest = Rs.89.83 and closing balance = Rs.2,581.11; month 5 interest = Rs.51.62 and amount due = Rs.2,632.73, so the final payment is Rs.2,000 and balance is Rs.632.73. The loan is not yet paid off; month 6 interest = Rs.12.65, so final payment is Rs.645.38. Total interest is approximately Rs.645.38.

```
#include <stdio.h>

int main(void) {
    double balance = 10000.0;
    const double rate = 0.02;
    const double repayment = 2000.0;
    double total_interest = 0.0;
    int month = 0;

    printf("Month  Opening    Interest  Payment  Closing\n");
    while (balance > 0.005 && month < 360) {
        double interest, due, payment;
        month++;
        interest = balance * rate;
        due = balance + interest;
        payment = (due < repayment) ? due : repayment;
        balance = due - payment;
        total_interest += interest;
        if (balance < 0.005) balance = 0.0;
        printf("%2d    %8.2f   %7.2f  %7.2f  %8.2f\n",
               month, due - interest, interest, payment, balance);
    }

    if (balance == 0.0)
        printf("Paid in %d months; total interest = Rs.%.2f\n",
               month, total_interest);
    else
        printf("Safety limit reached; balance = Rs.%.2f\n", balance);
    return 0;
}
```

```
Q6 - Loan repayment schedule

$ ./q6
Month  Opening    Interest  Payment  Closing
 1    10000.00    200.00  2000.00   8200.00
 2     8200.00    164.00  2000.00   6364.00
 3     6364.00    127.28  2000.00   4491.28
 4     4491.28     89.83  2000.00   2581.11
 5     2581.11     51.62  2000.00    632.73
 6      632.73     12.65   645.38      0.00
Paid in 6 months; total interest = Rs.645.38
```

Termination and safety

Do not compare a floating-point balance using balance != 0. Use a small tolerance such as balance > 0.005, or store money as integer paise. If repayment is less than or equal to monthly interest, the balance may not decrease; detect this condition and report that the loan will not be repaid under the chosen terms. A 360-month limit protects against an endless schedule. The closed-form formula may estimate the number of periods in O(1), but it requires careful handling of rounding and the final partial payment; simulation is O(m) for m months and directly produces the schedule.

Q7 – Diagnose, Redesign and Validate

Selected program: Q5 prime-number checker

A. Diagnose the bottleneck

The original loop checks every integer divisor from 2 through the square-root boundary. For 1,000,003, it performs 999 divisibility checks. Most of these checks are for even divisors after 2, even though an odd number cannot be divisible by an even number.

B. Measurable objective

Reduce the divisibility checks for the large prime 1,000,003 from 999 to about 500 while returning the same prime/not-prime answer.

C. Revised solution

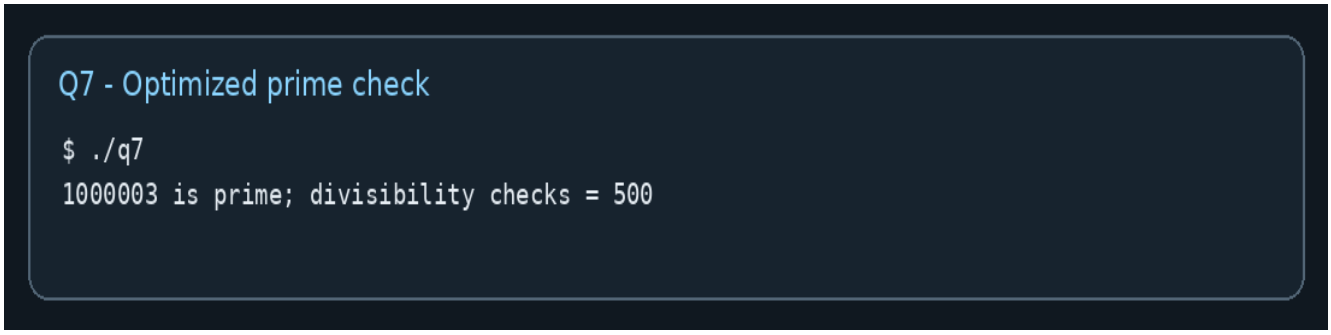
Check divisor 2 once. If n is even, return not prime. Otherwise test only odd divisors beginning at 3, increasing by 2. Use i <= n / i rather than i * i <= n to avoid multiplication overflow. No array or other additional memory is needed.

```
#include <stdio.h>

unsigned long checks = 0;

/* Improved trial division: test 2 once, then only odd divisors. */
int is_prime_optimized(unsigned long n) {
    checks = 0;
    if (n < 2) return 0;
    if (n == 2) return 1;
    if (n % 2 == 0) {
        checks++;
        return 0;
    }
    checks++; /* the divisibility test by 2 */
    for (unsigned long i = 3; i <= n / i; i += 2) {
        checks++;
        if (n % i == 0) return 0;
    }
    return 1;
}

int main(void) {
    unsigned long n = 1000003;
    int result = is_prime_optimized(n);
    printf("%lu is %s; divisibility checks = %lu\n",
           n, result ? "prime" : "not prime", checks);
    return 0;
}
```



Trade-off and conclusion

The optimized method retains O(√n) worst-case time and O(1) extra space, but roughly halves the number of trial divisions for large odd primes. It adds a small special-case check for even numbers. The redesign improves the actual work performed without merely changing the loop syntax.

Submission note

This PDF is the combined assignment document. Upload the same final PDF to both GitHub and the SIMATS submission platform; verify that each upload is complete before submission.